Привет,я пишу собственнную библиотеку содержащею функции(компоненты)для обучения LLM, которые могут выполняться на OpenCL.

Нам будт нужны 2 уровня абстракции:
1- nnclglob будет содержать самостятельные операции (такие как перемножение матриц ,softmax , и другие тяжелые функции которые требуют синхронизации либо расчета данных на строне CPU (к примеру персчитать размер новой матрицы приумножение матриц Y=X*D,D=Y*z1,X=D*W меняя местами указатели на буферы памяти и только перезаписывать буфер размерностей осей (тогда понадобиться всего 3 буфера для всех операций вместо 9 буферов размерностей для выполнения монолитным куском) + внутри глобальных абстракций можно на встроенном уровне добавлять в конец синхронизацию OpenCL чтобы данные были актульны.
Также нам будут нужны два спецефических глобальных объекта:
1. LocalRunner — запускает цепочку лоальных вызовов , преобразуя код в 1 opencl ядро.
2.HostReturner — переводит запрашиваемые данные с gpu на cpu (полезно для отладки).
На уровне LocalRunner мы будем использовать другой псевдокод содержащий nncllloc (сюда входят функции из файла „base_cl_functions“ (синус, косиснус и т. д.). LocalRunner также будет иметиь встроенную синхронизацию по завершению выполнения (чтобы гарантировать актуальность данных).
Таким образоми, псевдокод будет выглядеть притмерног так:

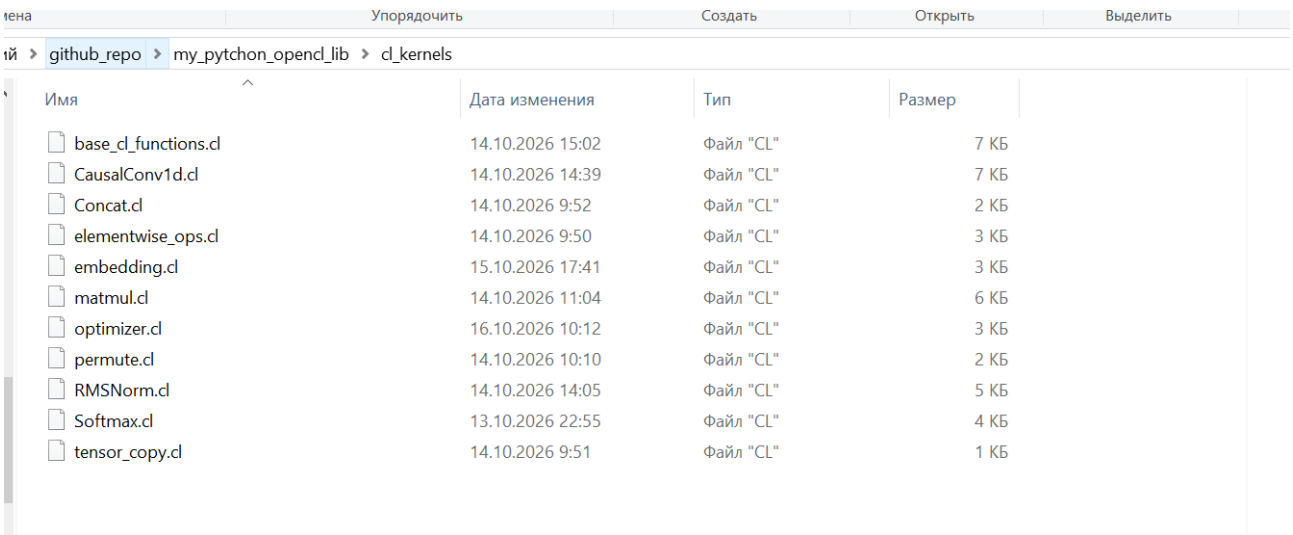
```
gate_A, gate_B, reader_C = nnclglob.split(projected, [self.d_state, self.d_state, self.d_model], dim=-1)

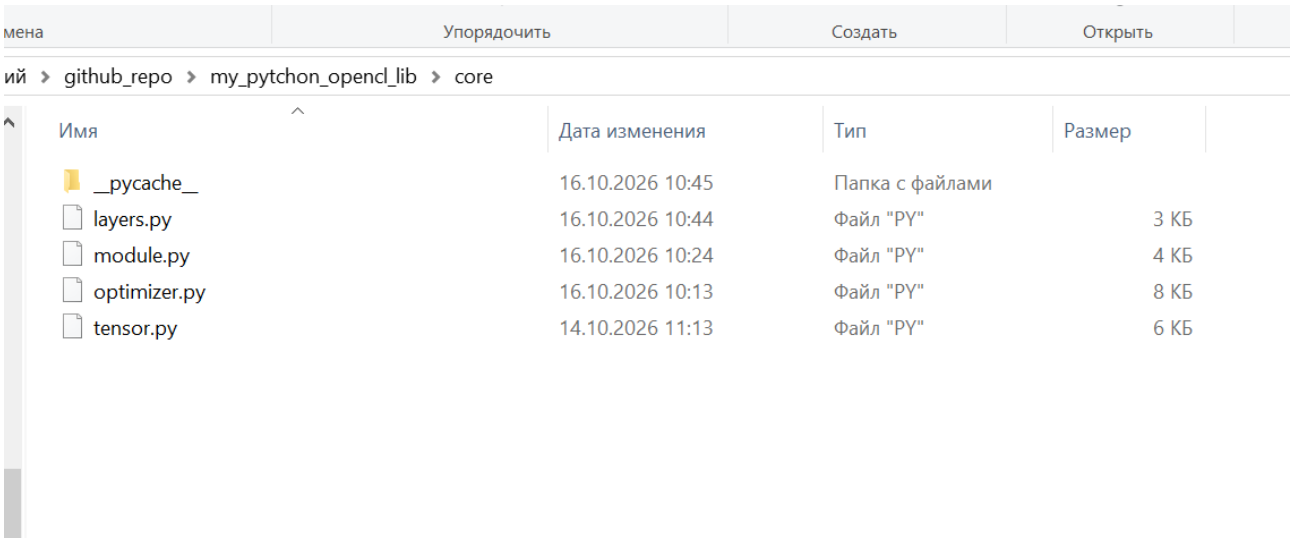
nnclglob. LocalRunner(()=>
        get_A=nncllloc.sigmoid(get_A)
        get_B=nncllloc.tanh(get_B)

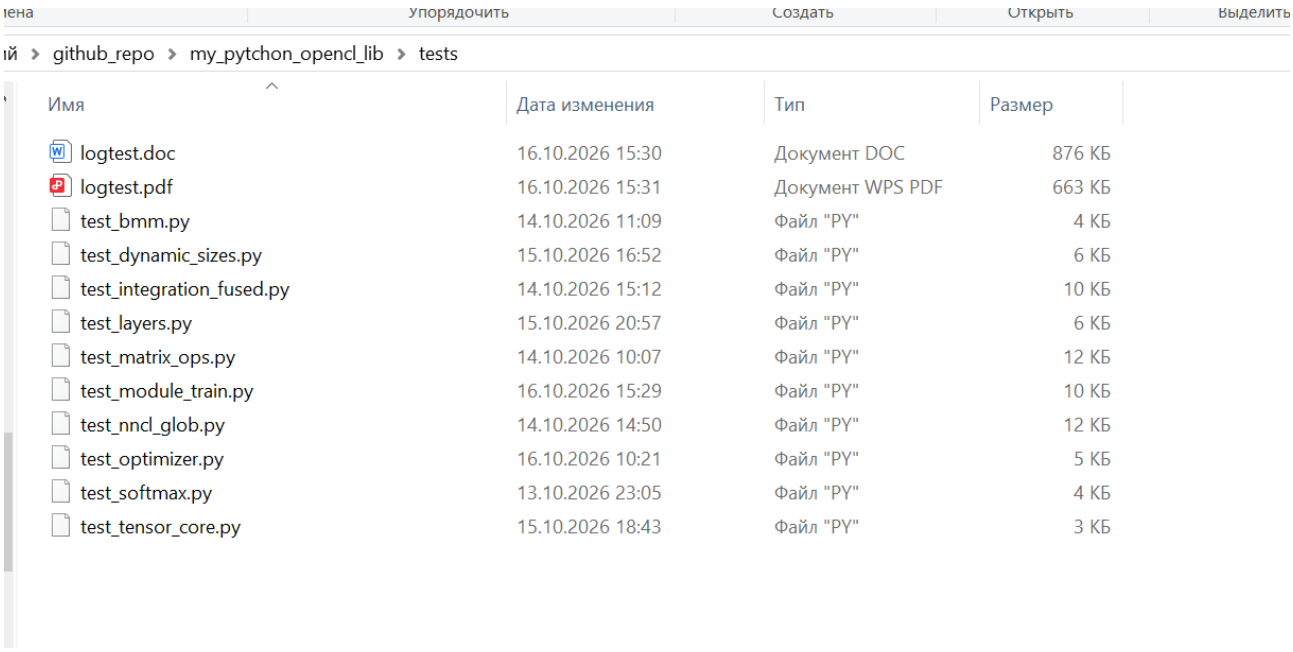
)
new_information = nnclglob.bmm(v, gate_B)
nnclglob. LocalRunner(()=>
        memory_state=memory_state*get_A
        memory_state=memory_state+new_information
)
tmp=nnclglob.transpose( memory_state,1, 2)
current_output=nnclglob.bmm(tmp,reader_C)
```

Решение проблемы обратного проохода

Чтобы решить проблему обратиного прохода, можно параллельно с основной цепочкой (при включенном реждиме обучения) строить 2 цепочку обратного прохода , заменяя функции их производными.







Большая часть компонентов библиотеки работает стабильно, но возникли проблемы с loaclRunner.

Каждый модуль покрывапеться тестами, проблемы возхникли именно с localCompiler.

Лог тестирования

PS C:\Users\Дмитрий\github_repo\my_pytchon_opencl_lib\tests> py -3.13 test_module_train.py

==========

🚀 ЗАПУСК С КВОЗНОГО ИНТЕГРАЦИОННОГО ТЕСТА ОБУЧЕНИЯ МОДЕЛИ (VRAM PIPELINE)

==========

Тренировка пайплайна на устройстве: NVIDIA GeForce RTX 2060

Шаг 1: Запуск Forward прохода всей модели...

C:\Users\Дмитрий\github_repo\my_pytchon_opencl_lib\compiler\nncl_glob.py:133: RepeatedKernelRetrieval: Kernel 'bmm_tiled' has been retrieved more than once. Each retrieval creates a new, independent kernel, at possibly considerable expense. To avoid the expense, reuse the retrieved kernel instance. To avoid this warning, use cl.Kernel(prg, name).

self.prg_matmul.bmm_tiled(

……

🔍 ТРАССИРОВКА ТЕКСТОВОГО JIT-БЛОКА (RUNNER LOG):

……

Обнаруженные объекты в области видимости forward():

- [Tensor] x -> size: 32, shape: [1, 4, 8]
- [Tensor] memory_state -> size: 32, shape: [4, 8]
- [Scalar] DYNAMIC_SEED -> value: 77777
- [Scalar] DROPOUT_P -> value: 0.2
- [Tensor] gate_vectors_3d -> size: 32, shape: [1, 4, 8]
- [Tensor] input_vectors_3d -> size: 32, shape: [1, 4, 8]
- [Tensor] gate_vectors -> size: 32, shape: [4, 8]
- [Tensor] input_vectors -> size: 32, shape: [4, 8]
- [Tensor] gate_A -> size: 32, shape: [4, 8]
- [Tensor] gate_B -> size: 32, shape: [4, 8]
- [Tensor] output -> size: 32, shape: [4, 8]

Компиляция строк выражений в Си-код:

- gate_A = nncllloc.sigmoid(gate_vectors)
- gate_B = nncllloc.tanh(input_vectors)
- memory_state = memory_state * gate_A
- memory_state = memory_state + gate_B
- output = nncllloc.dropout(memory_state, 0.2, tid, 77777)

……

C:\Users\Дмитрий\AppData\Local\Programs\Python\Python313\Lib\site-packages\pyopencl__init__.py:581: CompilerWarning: Non-empty compiler output encountered. Set the environment variable PYOPENCL_COMPILER_OUTPUT=1 to see more.

lambda: self._prg.build(options_bytes, devices),

Шаг 2: Имитация прихода градиента ошибки сверху (Loss Backward)...

результат:[[1. 1. 1. 1. 1. 1. 1. 1.]

[1. 1. 1. 1. 1. 1. 1. 1.]

[1. 1. 1. 1. 1. 1. 1. 1.]

[1. 1. 1. 1. 1. 1. 1. 1.]]

Шаг 3: Развертывание автоматического обратного прохода градиентов...

Шаг 4: Обновление параметров во VRAM через AdamWOptimizer...

C:\Users\Дмитрий\github_repo\my_pytchon_opencl_lib\core\optimizer.py:150: RepeatedKernelRetrieval: Kernel 'adamw_step_kernel' has been retrieved more than once. Each retrieval creates a new, independent kernel, at possibly considerable expense. To avoid the expense, reuse the retrieved kernel instance. To avoid this warning, use cl.Kernel(prg, name).

self.program.adamw_step_kernel(

Вычисление параллельного эталона на PyTorch для сверки...

Сверка результатов:

Traceback (most recent call last):

File "C:\Users\Дмитрий\github_repo\my_pytchon_opencl_lib\tests\test_module_train.py", line 195, in <module>

test_full_training_loop_step()

~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~^^

File "C:\Users\Дмитрий\github_repo\my_pytchon_opencl_lib\tests\test_module_train.py", line 184, in test_full_training_loop_step

np.testing.assert_allclose(cl_grad_w_gate, t_gate_proj.weight.grad.numpy(), rtol=1e-4, atol=1e-4)

~~~~~~~~~~~~~~~~~~~~~~~~~~~~^^^

First 5 mismatches are at indices:

[0, 0]: 0.0 (ACTUAL), -0.451382577419281 (DESIRED)

[0, 1]: 0.0 (ACTUAL), 0.25324511528015137 (DESIRED)

[0, 2]: 0.0 (ACTUAL), -0.5315306782722473 (DESIRED)

[0, 3]: 0.0 (ACTUAL), -0.009077847003936768 (DESIRED)

[0, 4]: 0.0 (ACTUAL), 0.2978948652744293 (DESIRED)

Max absolute difference among violations: 0.81730366

Max relative difference among violations: 1.

ACTUAL: array([[0., 0., 0., 0., 0., 0., 0.],

[0., 0., 0., 0., 0., 0., 0.],

[0., 0., 0., 0., 0., 0., 0.],...

DESIRED: array([[-0.451383, 0.253245, -0.531531, -0.009078, 0.297895, -0.817304,

-0.184555, 0.114919],

[-0.423188, 0.209237, -0.54083 , 0.032587, 0.210326, -0.658905,...

PS C:\Users\Дмитрий\github_repo\my_pytchon_opencl_lib\tests>

Исходный код проблемных модулей

test_module_train.py

C: > Users > Дмитрий > github_repo > my_pytchon_opencl_lib > tests > test_module_train.py

```python
import os
import sys
import numpy as np
import pyopencl as cl
import torch

# Добавляем корневую директорию проекта в путь поиска модулей Python
sys.path.append(os.path.abspath(os.path.join(os.path.dirname(__file__), '..')))

from core.tensor import Tensor
from core.module import Module
from core.layers import Linear
from core.optimizer import AdamWOptimizer
from compiler.nncl_compiler import LocalRunner
from compiler.nncl_glob import NnClGlob
from compiler.host_returner import HostReturner

# Симулируем легкий математический модуль nnclloc для синтаксиса лямбды
class DummyNnclLoc:
    def sigmoid(self, x): return x
    def tanh(self, x): return x
nnclloc = DummyNnclLoc()

# ==============================================================================
# СБОРКА ТЕСТОВОЙ МОДЕЛИ
# ==============================================================================
class TrainableMemoryBlock(Module):
    def __init__(self, d_model, ctx, queue):
        super().__init__()
        self.ctx = ctx
        self.queue = queue
        self.nnclglob = NnClGlob(ctx, queue)
        self.runner = LocalRunner(ctx, queue, is_training=True)

        # Объявляем два полносвязных слоя (проекции гейтов)
        self.gate_proj = Linear(d_model, d_model, self.nnclglob)
        self.input_proj = Linear(d_model, d_model, self.nnclglob)

    def forward(self, x, memory_state, DYNAMIC_SEED, DROPOUT_P):
            # 1. Глобальные линейные проекции: [1, 4, 8] -> [1, 4, 8]
            gate_vectors_3d = self.gate_proj(x)
            input_vectors_3d = self.input_proj(x)

            # Убираем фиктивную ось батча 1 через БЕСПЛАТНЫЙ виртуальный squeeze,
            # чтобы перевести векторы из 3D [1, 4, 8] в плоскую 2D геометрию,
            # которая в точности совпадает с геометрией memory_state!
            gate_vectors = gate_vectors_3d.squeeze(0)
            input_vectors = input_vectors_3d.squeeze(0)

            # Выделяем целевые буферы во VRAM строго правильной формы [4, 8]
            shape_2d = memory_state.shape # Форма [4, 8]

            gate_A = Tensor(shape_2d, self.ctx, self.queue)
            gate_B = Tensor(shape_2d, self.ctx, self.queue)
            output = Tensor(shape_2d, self.ctx, self.queue)

```

```
            gate_A = Tensor(shape_2d, self.ctx, self.queue)
            gate_B = Tensor(shape_2d, self.ctx, self.queue)
            output = Tensor(shape_2d, self.ctx, self.queue)

            gate_A.require_grad()
            gate_B.require_grad()
            output.require_grad()

            # 2. ТОТАЛЬНЫЙ МОНОЛИТНЫЙ JIT-ФЬЮЗИНГ (Вся математика гейтов за 1 проход по VRAM!)
            # Передаем обычную, красивую текстовую строку инструкций.
            # Наш LocalRunner отфильтрует только нужные операнды, автоматически вплавит константы
            # и скомпилирует идеальное Си-ядро.
            self.runner(f"""
                gate_A = nnclloc.sigmoid(gate_vectors)
                gate_B = nnclloc.tanh(input_vectors)
                memory_state = memory_state * gate_A
                memory_state = memory_state + gate_B
                output = nnclloc.dropout(memory_state, {DROPOUT_P}, tid, {DYNAMIC_SEED})
            """)

            # Извлекаем обновленные тензоры обратно
            return self.runner.get_tensor('output'), self.runner.get_tensor('memory_state')




def test_full_training_loop_step():
    print("\n" + "="*80)
    print("🚀 ЗАПУСК С КВОЗНОГО ИНТЕГРАЦИОННОГО ТЕСТА ОБУЧЕНИЯ МОДЕЛИ (VRAM PIPELINE)")
    print("="*80)

    # Инициализация единого OpenCL окружения
    platforms = cl.get_platforms()
    devices = platforms[0].get_devices()[0]
    ctx = cl.Context([devices])
    queue = cl.CommandQueue(ctx)
    print(f"Тренировка пайплайна на устройстве: {devices.name}")

    # Геометрия модели: Batch=1, Length=4, d_model=8
    B, L, D = 1, 4, 8

    np.random.seed(42)
    init_x = np.random.randn(B, L, D).astype(np.float32)
    init_memory = np.ones((B * L, D), dtype=np.float32) # Исходное состояние памяти

    # Параметры Dropout и Оптимизатора
    SEED = 77777
    P_DROP = 0.2
    LR = 0.01

    # ---------------------------------------------------------------------------
    # КОНТУР НАШЕЙ OPENCL БИБЛИОТЕКИ
    # ---------------------------------------------------------------------------
    x_cl = Tensor.from_numpy(init_x, ctx, queue)
    memory_cl = Tensor.from_numpy(init_memory, ctx, queue)
```

```python
    # КОНТУР НАШЕЙ OPENCL БИБЛИОТЕКИ
    # ------------------------------------------------------------------------
    x_cl = Tensor.from_numpy(init_x, ctx, queue)
    memory_cl = Tensor.from_numpy(init_memory, ctx, queue)
    memory_cl.require_grad()

    # Создаем модель и вытаскиваем копии весов для PyTorch до шага оптимизации
    model = TrainableMemoryBlock(d_model=D, ctx=ctx, queue=queue)
    w_gate_init = HostReturner.to_cpu(model.gate_proj.weight)
    w_in_init = HostReturner.to_cpu(model.input_proj.weight)

    # Инициализируем AdamW, автоматически собирая параметры через Module.parameters()
    optimizer = AdamWOptimizer(model.parameters(), lr=LR, betas=(0.9, 0.999), weight_decay=0.0)

    print("\nШаг 1: Запуск Forward прохода всей модели...")
    output_cl, final_memory_cl = model(x_cl, memory_cl, DYNAMIC_SEED=SEED, DROPOUT_P=P_DROP)

    print("Шаг 2: Имитация прихода градиента ошибки сверху (Loss Backward)...")
    # Пусть от функции потерь пришел единичный вектор градиента ошибки по выходу
    grad_output = np.ones((B * L, D), dtype=np.float32)
    cl.enqueue_copy(queue, output_cl.grad_buffer, grad_output)
    queue.finish()
    print(f"результат:{grad_output}")
    print("Шаг 3: Развертывание автоматического обратного прохода градиентов...")
    model.runner.run_backward()

    # Скачиваем вычисленные градиенты для проверки
    cl_grad_w_gate = HostReturner.to_cpu_grad(model.gate_proj.weight)

    print("Шаг 4: Обновление параметров во VRAM через AdamWOptimizer...")
    optimizer.step()

    # Скачиваем обновленные веса для сравнения
    cl_updated_w_gate = HostReturner.to_cpu(model.gate_proj.weight)

    # ------------------------------------------------------------------------
    # ЭТАЛОННЫЙ КОНТУР В PYTORCH
    # ------------------------------------------------------------------------
    print("\nВычисление параллельного эталона на PyTorch для сверки...")
    t_x = torch.tensor(init_x, requires_grad=True)
    t_memory = torch.tensor(init_memory, requires_grad=True)

    t_gate_proj = torch.nn.Linear(D, D, bias=False)
    t_input_proj = torch.nn.Linear(D, D, bias=False)

    with torch.no_grad():
        t_gate_proj.weight.copy_(torch.from_numpy(w_gate_init))
        t_input_proj.weight.copy_(torch.from_numpy(w_in_init))

    # Прямой ход PyTorch (с выпрямлением, как в нашей bmm-реализации)
    t_x_2d = t_x.reshape(-1, D)
    t_gate_vecs = t_gate_proj(t_x_2d)
    t_input_vecs = t_input_proj(t_x_2d)

    t_gate_A = torch.sigmoid(t_gate_vecs)
    t_gate_B = torch.tanh(t_input_vecs)
```

```
    t_gate_A = torch.sigmoid(t_gate_vecs)
    t_gate_B = torch.tanh(t_input_vecs)

    t_mem_fused = t_memory * t_gate_A + t_gate_B

    # Имитируем наш GPU детерминированный Dropout через маску выходов
    cl_output_np = HostReturner.to_cpu(output_cl)
    mask = (cl_output_np != 0.0).astype(np.float32)
    t_output = t_mem_fused * torch.from_numpy(mask) / (1.0 - P_DROP)

    # Обратный ход PyTorch
    t_output.backward(torch.ones((B * L, D)))

    # Шаг эталонного оптимизатора AdamW
    t_optimizer = torch.optim.AdamW(
        [t_gate_proj.weight, t_input_proj.weight], lr=LR, betas=(0.9, 0.999), weight_decay=0.0
    )
    t_optimizer.step()

    # ---------------------------------------------------------------------
    # ФИНАЛЬНАЯ ВЕРИФИКАЦИЯ (ASSERT_ALLCLOSE)
    # ---------------------------------------------------------------------
    print("\nСверка результатов:")

    # 1. Проверяем градиенты до обновления
    np.testing.assert_allclose(cl_grad_w_gate, t_gate_proj.weight.grad.numpy(), rtol=1e-4, atol=1e-4)
    print("  ✅ Шаг Градиентов: Градиенты тяжелых линейных слоев сошлись с PyTorch!")

    # 2. Проверяем обновленные веса во VRAM
    np.testing.assert_allclose(cl_updated_w_gate, t_gate_proj.weight.detach().numpy(), rtol=1e-4, atol=1e-4)
    print("  ✅ Шаг Оптимизации: Обновленные веса AdamW во VRAM совпали с PyTorch!")

    print("\n🎉 ПОЛНОЦЕННЫЙ ЦИКЛ ОБУЧЕНИЯ ВЕРИФИЦИРОВАН И УСПЕШНО ПРОЙДЕН!")
    print("="*80 + "\n")

if __name__ == "__main__":
    test_full_training_loop_step()
```

module.py

test_module_train.py | module.py

C: > Users > Дмитрий > github_repo > my_pytchon_opencl_lib > core > module.py

```python
from core.tensor import Tensor

class Module:
    def __init__(self):
        self._modules = {}
        self.training = True

    def __setattr__(self, name, value):
        """
        Автоматически перехватывает объявление подмодулей (например, self.linear = Linear(...))
        и сохраняет ссылки на них в системный словарь _modules.
        """
        if isinstance(value, Module) or hasattr(value, 'forward'):
            self.__dict__['_modules'][name] = value
        super().__setattr__(name, value)

    def train(self, mode: bool = True):
        """Переключает модель и все её подмодули в режим обучения/инференса"""
        self.training = mode
        for module in self._modules.values():
            if hasattr(module, 'train'):
                module.train(mode)
        # Переключаем также глобальный и локальный ранеры, если они привязаны
        if hasattr(self, 'runner') and self.runner is not None:
            self.runner.is_training = mode

    def eval(self):
        """Переключает модель в режим инференса (тестирования)"""
        self.train(mode=False)

    def parameters(self):
        """
        Рекурсивно обходит всю модель, собирает и возвращает генератор
        всех обучаемых параметров (объектов Tensor, у которых включен require_grad)
        """
        params = []

        # 1. Ищем тензоры внутри текущего модуля
        for name, attr in self.__dict__.items():
            if isinstance(attr, Tensor) and attr.grad_buffer is not None:
                if attr not in params:
                    params.append(attr)

        # 2. Рекурсивно лезем во все дочерние подмодули
        for name, module in self._modules.items():
            # Если это кастомный слой (например, Linear или Embedding), вытаскиваем его веса
            for attr_name, attr_val in module.__dict__.items():
                if isinstance(attr_val, Tensor):
                    # Если у тензора лениво не выделен grad_buffer, но это обучаемый вес
                    if attr_name in ['weight', 'bias']:
                        attr_val.require_grad()
                    if attr_val not in params:
                        params.append(attr_val)

            # Если дочерний модуль сам унаследован от Module, вызываем его параметры
            if isinstance(module, Module):
```

```python
        for attr_name, attr_val in module.__dict__.items():
            if isinstance(attr_val, Tensor):
                # Если у тензора лениво не выделен grad_buffer, но это обучаемый вес
                if attr_name in ['weight', 'bias']:
                    attr_val.require_grad()
                if attr_val not in params:
                    params.append(attr_val)

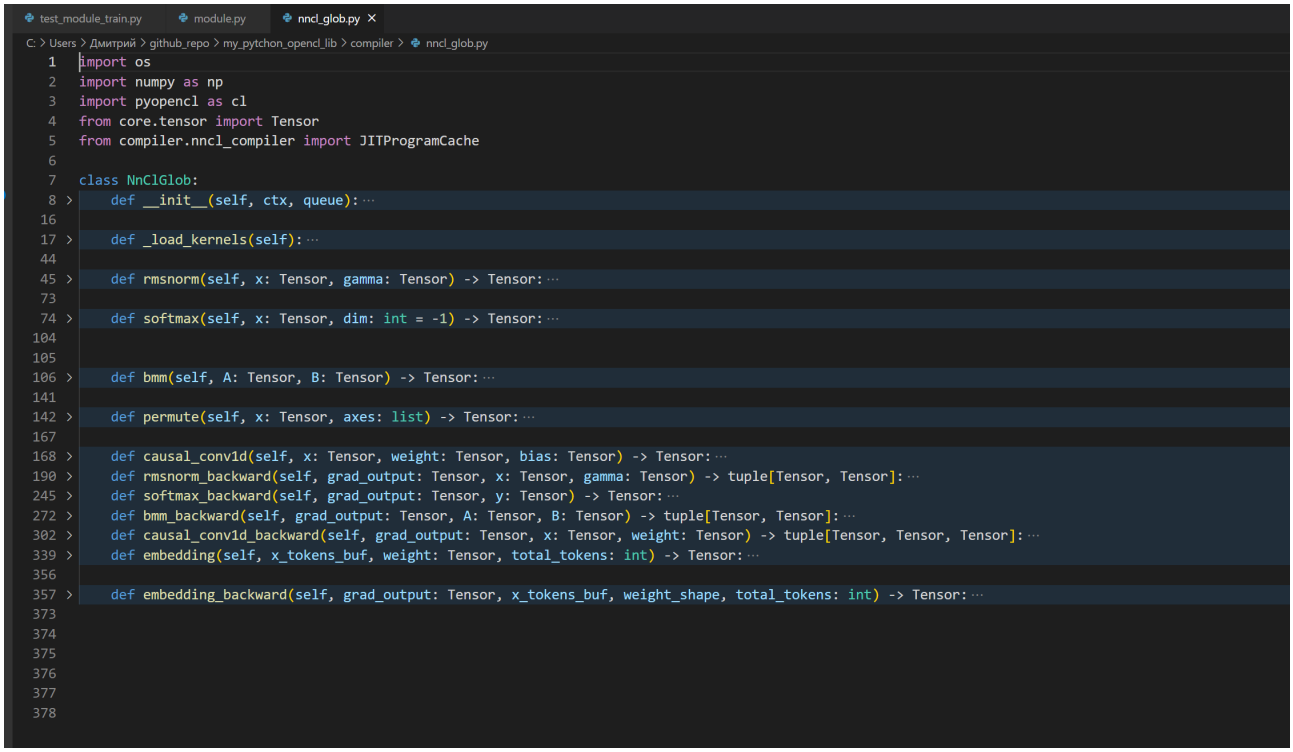
        # Если дочерний модуль сам унаследован от Module, вызываем его параметры
        if isinstance(module, Module):
            for p in module.parameters():
                if p not in params:
                    params.append(p)

    return params

def __call__(self, *args, **kwargs):
    """Позволяет вызывать модуль как функцию: model(x) вместо model.forward(x)"""
    return self.forward(*args, **kwargs)

def forward(self, *args, **kwargs):
    raise NotImplementedError("Каждый модуль обязан реализовать свой метод forward()!")
```

nncl_glob (кратко для справки (код тесты прошел))



test_module_train.py | module.py | nncl_glob.py

C: > Users > Дмитрий > github_repo > my_pytchon_opencl_lib > compiler > nncl_glob.py

```python
import os
import numpy as np
import pyopencl as cl
from core.tensor import Tensor
from compiler.nncl_compiler import JITProgramCache

class NnClGlob:
    def __init__(self, ctx, queue): ···

    def _load_kernels(self): ···

    def rmsnorm(self, x: Tensor, gamma: Tensor) -> Tensor: ···

    def softmax(self, x: Tensor, dim: int = -1) -> Tensor: ···

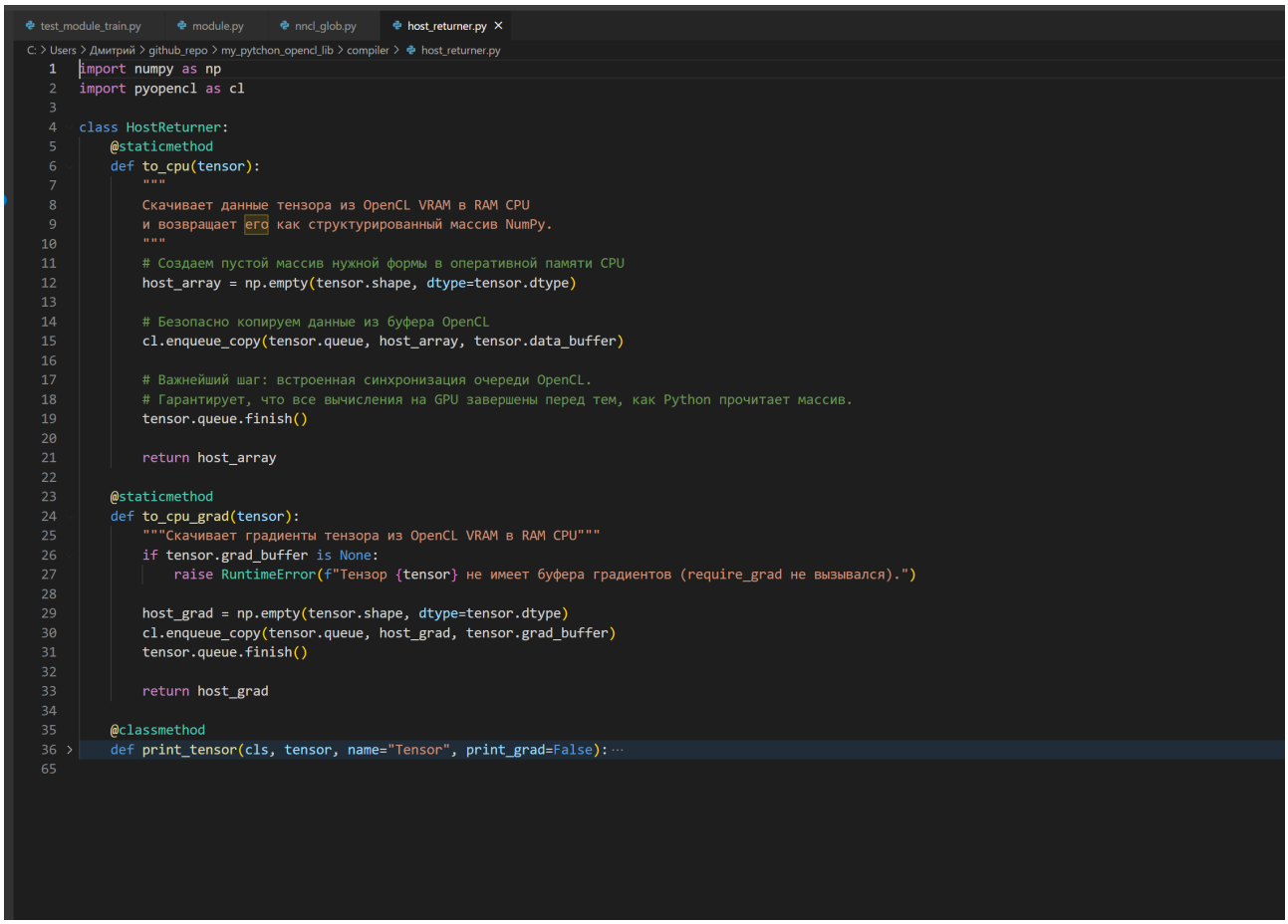

    def bmm(self, A: Tensor, B: Tensor) -> Tensor: ···

    def permute(self, x: Tensor, axes: list) -> Tensor: ···

    def causal_conv1d(self, x: Tensor, weight: Tensor, bias: Tensor) -> Tensor: ···
    def rmsnorm_backward(self, grad_output: Tensor, x: Tensor, gamma: Tensor) -> tuple[Tensor, Tensor]: ···
    def softmax_backward(self, grad_output: Tensor, y: Tensor) -> Tensor: ···
    def bmm_backward(self, grad_output: Tensor, A: Tensor, B: Tensor) -> tuple[Tensor, Tensor]: ···
    def causal_conv1d_backward(self, grad_output: Tensor, x: Tensor, weight: Tensor) -> tuple[Tensor, Tensor, Tensor]: ···
    def embedding(self, x_tokens_buf, weight: Tensor, total_tokens: int) -> Tensor: ···

    def embedding_backward(self, grad_output: Tensor, x_tokens_buf, weight_shape, total_tokens: int) -> Tensor: ···
```

nncl_hostReturner (аналогично тесты прошел)



test_module_train.py | module.py | nncl_glob.py | host_returner.py

C: > Users > Дмитрий > github_repo > my_pytchon_opencl_lib > compiler > host_returner.py

```python
import numpy as np
import pyopencl as cl

class HostReturner:
    @staticmethod
    def to_cpu(tensor):
        """
        Скачивает данные тензора из OpenCL VRAM в RAM CPU
        и возвращает его как структурированный массив NumPy.
        """
        # Создаем пустой массив нужной формы в оперативной памяти CPU
        host_array = np.empty(tensor.shape, dtype=tensor.dtype)

        # Безопасно копируем данные из буфера OpenCL
        cl.enqueue_copy(tensor.queue, host_array, tensor.data_buffer)

        # Важнейший шаг: встроенная синхронизация очереди OpenCL.
        # Гарантирует, что все вычисления на GPU завершены перед тем, как Python прочитает массив.
        tensor.queue.finish()

        return host_array

    @staticmethod
    def to_cpu_grad(tensor):
        """Скачивает градиенты тензора из OpenCL VRAM в RAM CPU"""
        if tensor.grad_buffer is None:
            raise RuntimeError(f"Тензор {tensor} не имеет буфера градиентов (require_grad не вызывался).")

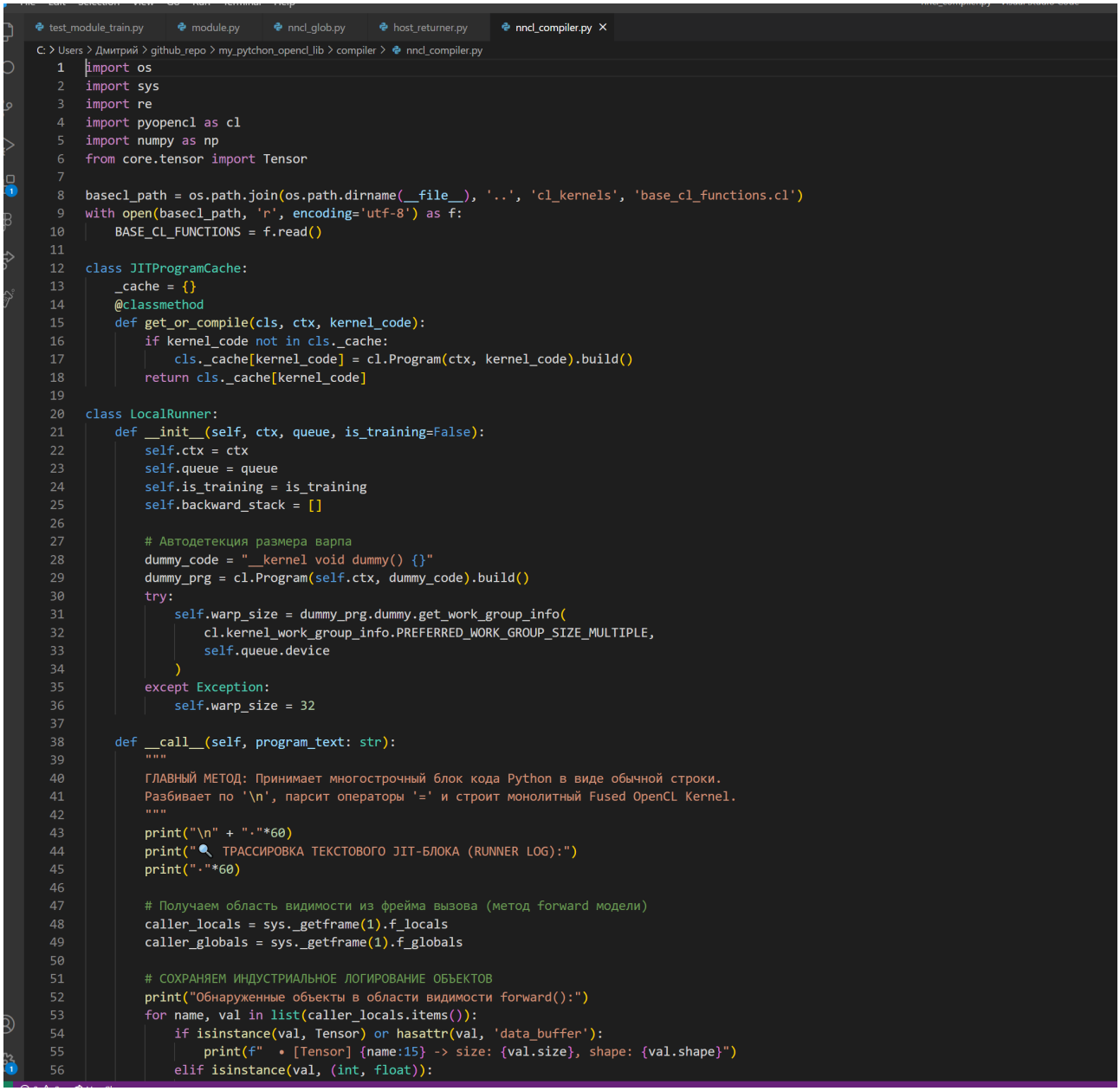
        host_grad = np.empty(tensor.shape, dtype=tensor.dtype)
        cl.enqueue_copy(tensor.queue, host_grad, tensor.grad_buffer)
        tensor.queue.finish()

        return host_grad

    @classmethod
    def print_tensor(cls, tensor, name="Tensor", print_grad=False): ···
```

LoaclRunner (проблемный модуль)

Компилирует переданный код в OpenCL ядро захватывая переменные и тензоры (openCL буфферы,линейные массивы содержащие параметры размерности). Как и оргинальный pytorch сохраняет исходные значения для градиентов обратного прохода.



```
        for name, val in list(caller_locals.items()):
            if isinstance(val, Tensor) or hasattr(val, 'data_buffer'):
                print(f"  • [Tensor] {name:15} -> size: {val.size}, shape: {val.shape}")
            elif isinstance(val, (int, float)):
                print(f"  • [Scalar] {name:15} -> value: {val}")

        # Разбиваем текст на отдельные чистые инструкции
        expressions = [e.strip() for e in program_text.strip().split('\n') if e.strip()]

        detected_tensors = {}
        detected_scalars = {}
        saved_tensors_vram = {}
        cl_body_forward = []
        cl_body_backward = []

        cl_keywords = ['float', 'int', 'tid', 'unsigned', 'p', 'seed', 'p_val', 'seed_val', 'DROPOUT_P', 'DYNAMIC_SEED']

        print("\nКомпиляция строк выражений в Си-код:")
        for expr in expressions:
            # Убираем возможные двойные '==', оставляя строго чистые '='
            expr_clean = expr.replace('==', '=')
            if '=' not in expr_clean: continue

            # Легко отделяем целевую переменную (target) от функции (operation)
            target, operation = expr_clean.split('=', 1)
            target, operation = target.strip(), operation.strip()
            print(f"  • {target:12} = {operation}")

            cl_op = operation.replace('nncl1oc.', 'nn_')
            while cl_op.count(')') > cl_op.count('('):
                last_bracket = cl_op.rfind(')')
                if last_bracket != -1: cl_op = cl_op[:last_bracket] + cl_op[last_bracket+1:]

            # ================================================================
            # ИЗОЛИРОВАННЫЙ БЛОК ДЛЯ DROPOUT С ДИНАМИЧЕСКИМИ РАЗМЕРАМИ
            # ================================================================
            if "nn_dropout" in cl_op:
                drop_match = re.search(r'nn_dropout\((.*?), (.*?), tid, (.*?)\)', cl_op)
                if drop_match:
                    inner_expr = drop_match.group(1).strip()
                    p_val = drop_match.group(2).strip()
                    seed_val = drop_match.group(3).strip()

                    tokens_inner = re.findall(r'[a-zA-Z_][a-zA-Z0-9_]*', inner_expr)
                    cl_inner_fwd = inner_expr
                    for t in tokens_inner:
                        if t not in ['DROPOUT_P', 'DYNAMIC_SEED'] and not t.isdigit():
                            val_obj = caller_locals.get(t, caller_globals.get(t, None))
                            if val_obj is not None and (isinstance(val_obj, Tensor) or hasattr(val_obj, 'data_buffer')):
                                detected_tensors[t] = True
                                cl_inner_fwd = re.sub(r'\b' + t + r'\b', f"{t}[tid]", cl_inner_fwd)
                            else:
                                detected_scalars[t] = True

                    detected_tensors[target] = True
                    cl_body_forward.append(f"if (tid < {target}Size) {{ {target}[tid] = nn_dropout({cl_inner_fwd}, {p_val}, tid, {seed_val}); }}")
```

0 ⚠ 0 Live Share

```
                    detected_scalars[t] = True

                detected_tensors[target] = True
                cl_body_forward.append(f"if (tid < {target}Size) {{ {target}[tid] = nn_dropout({cl_inner_fwd}, {p_val}, tid, {seed_val}); }}")

                if self.is_training:
                    base_target_tensor = inner_expr.split(',')[0].strip()
                    cl_body_backward.append(f"if (tid < {target}Size) {{")
                    cl_body_backward.append(f"        float upstream_{target} = grad_{target}[tid];")
                    cl_body_backward.append(f"        float back_drop_{target} = nn_dropout_backward(upstream_{target}, {p_val}, tid, {seed_val});")

                    if "*" in base_target_tensor:
                        u, v = [item.strip() for item in base_target_tensor.split('*')]
                        detected_tensors[u], detected_tensors[v] = True, True
                        cl_body_backward.append(f"        if (tid < {u}Size) grad_{u}[tid] += back_drop_{target} * {v}[tid];")
                        cl_body_backward.append(f"        if (tid < {v}Size) grad_{v}[tid] += back_drop_{target} * {u}[tid];")
                    elif "+" in base_target_tensor:
                        vars_add = [v.strip() for v in base_target_tensor.split('+')]

                        for v in vars_add:
                            detected_tensors[v] = True
                            cl_body_backward.append(f"        if (tid < {v}Size) grad_{v}[tid] += back_drop_{target};")
                    else:
                        detected_tensors[base_target_tensor] = True
                        cl_body_backward.append(f"        if (tid < {base_target_tensor}Size) grad_{base_target_tensor}[tid] += back_drop_{target};")
                    cl_body_backward.append(f"    }}")
                continue

            # ================================================================
            # СТАНДАРТНЫЙ БЛОК ОПЕРАЦИЙ (Сигмоиды, Произведения)
            # ================================================================
            tokens = re.findall(r'[a-zA-Z_][a-zA-Z0-9_]*', cl_op)
            if target in tokens and self.is_training:
                saved_name = f"saved_{target}"
                tensor_obj = caller_locals.get(target, caller_globals.get(target))
                vram_copy = cl.Buffer(self.ctx, cl.mem_flags.READ_WRITE, tensor_obj.nbytes)
                cl.enqueue_copy(self.queue, vram_copy, tensor_obj.data_buffer)
                saved_tensors_vram[saved_name] = vram_copy

            # Строка 143 остается как есть (начало цикла):
            for t in tokens:
                if t.startswith('nn_') or t in cl_keywords or t.isdigit(): continue
                val_obj = caller_locals.get(t, caller_globals.get(t, None))
                if val_obj is not None and (isinstance(val_obj, Tensor) or hasattr(val_obj, 'data_buffer')):
                    detected_tensors[t] = True
                    # СТРОКА 149 — ИСПРАВЛЕНИЕ (Корректное регулярное выражение замены на f-строку с [tid]):
                    cl_op = re.sub(r'\b' + t + r'\b', f"{t}[tid]", cl_op)
                else:
                    detected_scalars[t] = True

            detected_tensors[target] = True
            # СТРОКА 154 — ИСПРАВЛЕНИЕ (Убираем лишние фигурные скобки, мешающие Си-компиляции):
            cl_body_forward.append(f"if (tid < {target}Size) {{ {target}[tid] = {cl_op}; }}")


            if self.is_training:
```

```
            cl_body_forward.append(f" if (tid < {target}Size) {{ {target}[tid] = {cl_op}; }} ")


        if self.is_training:
            func_match = re.search(r'(nn_[a-zA-Z0-9_]+)\((.*?)\)', cl_op)
            if func_match:
                func_name = func_match.group(1)
                src_var = func_match.group(2).replace('[tid]', '').strip()
                use_src = f"saved_{src_var}" if f"saved_{src_var}" in saved_tensors_vram else src_var
                cl_body_backward.append(
                    f"if (tid < {target}Size) {{\n"
                    f"        float upstream_{target} = grad_{target}[tid];\n"
                    f"        if (tid < {src_var}Size) grad_{src_var}[tid] = {func_name}_backward({use_src}[tid], upstream_{target});\n"
                    f"    }}"
                )
            elif "*" in operation:
                u, v = [item.strip() for item in operation.split('*')]
                use_u = f"saved_{u}" if f"saved_{u}" in saved_tensors_vram else u
                use_v = f"saved_{v}" if f"saved_{v}" in saved_tensors_vram else v
                cl_body_backward.append(
                    f"if (tid < {target}Size) {{\n"
                    f"        float upstream_{target} = grad_{target}[tid];\n"
                    f"        if (tid < {u}Size) grad_{u}[tid] += upstream_{target} * {use_v}[tid];\n"
                    f"        if (tid < {v}Size) grad_{v}[tid] += upstream_{target} * {use_u}[tid];\n"
                    f"    }}"
                )
            elif any(op in operation for op in ["+", "-", "/"]):
                for t in tokens:
                    if t in detected_tensors and t != target:
                        cl_body_backward.append(
                            f"if (tid < {target}Size) {{\n"
                            f"        if (tid < {t}Size) grad_{t}[tid] += grad_{target}[tid];\n"
                            f"    }}"
                        )


        print("·"*60 + "\n")

        # Компиляция Си-кода
        kernel_hash = abs(hash(program_text))
        kernel_name = f"text_forward_{kernel_hash}"
        kernel_args = []
        # Объединяем весь сгенерированный Си-код в одну строку для поиска подстрок
        compiled_code_str = "\n".join(cl_body_forward) + "\n".join(cl_body_backward)

        for tensor in detected_tensors.keys():
            # ИСПРАВЛЕНИЕ: Проверяем, что имя тензора действительно используется в кодовой строке ядра!
            if re.search(r'\b' + tensor + r'\b', compiled_code_str):
                kernel_args.append(f"__global float* {tensor}")
                kernel_args.append(f"const int {tensor}Size")
                if self.is_training:
                    kernel_args.append(f"__global float* grad_{tensor}")

        for scalar in detected_scalars.keys():
            # ИСПРАВЛЕНИЕ: Проверяем использование скаляра
            if re.search(r'\b' + scalar + r'\b', compiled_code_str):
                kernel_args.append(f"const float {scalar}")
```

```
            kernel_args.append(f"const int {tensor}_size")
            if self.is_training:
                kernel_args.append(f"__global float* grad_{tensor}")

    for scalar in detected_scalars.keys():
        # ИСПРАВЛЕНИЕ: Проверяем использование скаляра
        if re.search(r'\b' + scalar + r'\b', compiled_code_str):
            kernel_args.append(f"const float {scalar}")

    for saved_name in saved_tensors_vram.keys():
        if re.search(r'\b' + saved_name + r'\b', compiled_code_str):
            kernel_args.append(f"__global const float* {saved_name}")
    args_str = ", ".join(kernel_args)
    forward_statements = "\n    ".join(cl_body_forward)
    full_forward_code = f"""
    {BASE_CL_FUNCTIONS}
    __kernel void {kernel_name}({args_str}) {{
    int tid = get_global_id(0) * {self.warp_size} + get_global_id(1);
    {forward_statements}
    }}
    """
    if 'DROPOUT_P' in caller_locals:
        full_forward_code = full_forward_code.replace('DROPOUT_P', f"{float(caller_locals['DROPOUT_P'])}f")
    if 'DYNAMIC_SEED' in caller_locals:
        full_forward_code = full_forward_code.replace('DYNAMIC_SEED', f"{int(caller_locals['DYNAMIC_SEED'])}")
    program = JITProgramCache.get_or_compile(self.ctx, full_forward_code)
    kernel = getattr(program, kernel_name)
    max_elements = 1
    cl_buffers_forward = []

    # === ВНУТРИ compiler/nncl_compiler.py (Сборка буферов для вызова ядра) ===
    max_elements = 1
    cl_buffers_forward = []

    for tensor in detected_tensors.keys():
        # ИСПРАВЛЕНИЕ: Передаем буфер на GPU только если он есть в кодовой строке ядра!
        if re.search(r'\b' + tensor + r'\b', compiled_code_str):
            tensor_obj = caller_locals.get(tensor, caller_globals.get(tensor))
            if tensor_obj.size > max_elements:
                max_elements = tensor_obj.size
            cl_buffers_forward.append(tensor_obj.data_buffer)
            cl_buffers_forward.append(np.int32(tensor_obj.size))
            if self.is_training:
                tensor_obj.require_grad()
                cl_buffers_forward.append(tensor_obj.grad_buffer)

    for scalar in detected_scalars.keys():
        # ИСПРАВЛЕНИЕ: Передаем скаляр только если он есть в кодовой строке ядра!
        if re.search(r'\b' + scalar + r'\b', compiled_code_str):
            scalar_val = caller_locals.get(scalar, caller_globals.get(scalar))
            cl_buffers_forward.append(np.float32(scalar_val))

    cl_buffers_call_fwd = list(cl_buffers_forward)
    for saved_name, vram_buffer in saved_tensors_vram.items():
        # ИСПРАВЛЕНИЕ: Проверяем сохраненные in-place кэши
        if re.search(r'\b' + saved_name + r'\b', compiled_code_str):
            cl_buffers_call_fwd.append(vram_buffer)
```

```
        for saved_name, vram_buffer in saved_tensors_vram.items():
            # ИСПРАВЛЕНИЕ: Проверяем сохраненные in-place кэши
            if re.search(r'\b' + saved_name + r'\b', compiled_code_str):
                cl_buffers_call_fwd.append(vram_buffer)



        total_count = int(np.ceil(max_elements / float(self.warp_size)) * self.warp_size)
        global_size = (total_count // self.warp_size, self.warp_size)
        kernel(self.queue, global_size, None, *cl_buffers_call_fwd)
        self.queue.finish()
        if self.is_training and cl_body_backward:
            backward_statements = "\n    ".join(cl_body_backward)
            backward_kernel_name = f"text_backward_{kernel_hash}"
            full_backward_code = f"""
            {BASE_CL_FUNCTIONS}
            __kernel void {backward_kernel_name}({args_str}) {{
            int tid = get_global_id(0) * {self.warp_size} + get_global_id(1);
            {backward_statements}
            }}
            """
            if 'DROPOUT_P' in caller_locals:
                full_backward_code = full_backward_code.replace('DROPOUT_P', f"{float(caller_locals['DROPOUT_P'])}f")
            if 'DYNAMIC_SEED' in caller_locals:
                full_backward_code = full_backward_code.replace('DYNAMIC_SEED', f"{int(caller_locals['DYNAMIC_SEED'])}")
            cl_buffers_backward = list(cl_buffers_forward)

            for saved_name, vram_buffer in saved_tensors_vram.items():
                if re.search(r'\b' + saved_name + r'\b', compiled_code_str):
                    cl_buffers_backward.append(vram_buffer)

            self.backward_stack.append((full_backward_code, backward_kernel_name, cl_buffers_backward, global_size))


    def run_backward(self):
        if not self.backward_stack:
            return
        while self.backward_stack:
            code, name, buffers, global_size = self.backward_stack.pop()
            program = JITProgramCache.get_or_compile(self.ctx, code)
            kernel = getattr(program, name)
            kernel(self.queue, global_size, None, *buffers)
            self.queue.finish()
    def get_tensor(self, name: str) -> Tensor:
            """
            Возвращает объект Tensor по его текстовому имени,
            который участвовал в вычислениях последнего JIT-ядра.
            """
            # Сначала ищем в сохраненном кэше локальных переменных вызова модели
            caller_locals = getattr(self, '_active_caller_locals', None)
            if caller_locals is None:
                # Если кэша нет, заглядываем на один фрейм вверх напрямую
                caller_locals = sys._getframe(1).f_locals

            val_obj = caller_locals.get(name, None)
            if val_obj is None:
```

```
            self.queue.finish()
    def get_tensor(self, name: str) -> Tensor:
            """
            Возвращает объект Tensor по его текстовому имени,
            который участвовал в вычислениях последнего JIT-ядра.
            """
            # Сначала ищем в сохраненном кэше локальных переменных вызова модели
            caller_locals = getattr(self, '_active_caller_locals', None)
            if caller_locals is None:
                # Если кэша нет, заглядываем на один фрейм вверх напрямую
                caller_locals = sys._getframe(1).f_locals

            val_obj = caller_locals.get(name, None)
            if val_obj is None:
                raise NameError(f"Тензор с именем '{name}' не найден в области видимости вызова forward().")
            return val_obj
```